Layered Fluid Simulation with MPM

Sarah Etter

April 2026

1 Overview & Methodology

Project Aim

The goal of this project was to implement a miniature version of Grant Kot's interactive fluid simulation *Liquid Layers* [1], which simulates multiple colored viscoelastic fluids using the particle-based method of Clavet et al. [2]. The demo allows the user to manipulate material properties and interactions including gravity, inter-particle forces, attraction/repulsion, and particle emission and deletion. As shown in Fig. 1, the simulation exhibits clear density stratification and coherent multi-material motion. My goal was not to exactly reproduce Kot's particle-based method, but to create a related layered-fluid simulation using MPM, a hybrid Lagrangian/Eulerian continuum simulation framework in which particles carry material state while a background grid computes forces and spatial derivatives.

MPM: Theory and Derivation

The mathematical problem in MPM is to simulate the time evolution of a continuum material. Given an initial configuration, we seek the material's position, velocity, density, deformation, and internal stress at later times. This evolution is governed by conservation of mass and conservation of momentum:

$$\frac{D\rho}{Dt} + \rho\nabla\cdot\mathbf{v} = 0, \qquad \rho\frac{D\mathbf{v}}{Dt} = \nabla\cdot\boldsymbol{\sigma} + \mathbf{f}_b. \tag{1}$$

Here ρ is density, $\mathbf{v}$ is velocity, D/Dt is the material derivative, $\boldsymbol{\sigma}$ is the Cauchy stress tensor, and $\mathbf{f}_b$ represents body forces such as gravity. The first equation states that mass is neither created nor destroyed. The second is Newton's second law for a continuum: mass times acceleration equals net force from internal stress plus external body forces.

MPM represents the material using Lagrangian particles. Each particle p carries a fixed mass m_p, position $\mathbf{x}_p$, velocity $\mathbf{v}_p$, volume V_p, deformation gradient $\mathbf{F}_p$, and any material-specific state variables. Because each particle has constant mass, mass conservation is automatically satisfied. The main task is therefore to solve the momentum equation.

The momentum equation is not closed until we specify how stress depends on the material state. This is the role of the constitutive model, which gives stress as a function of deformation and material parameters: $\boldsymbol{\sigma} = \boldsymbol{\sigma}(\mathbf{F}, \dot{\mathbf{F}}, \ldots)$. For example, a fluid model may generate pressure from compression, while an elastic solid model generates stress from stretching and shearing.

The momentum equation in strong form, $\rho\, D\mathbf{v}/Dt - \nabla\cdot\boldsymbol{\sigma} - \mathbf{f}_b = 0$, requires pointwise evaluation of the stress divergence $\nabla\cdot\boldsymbol{\sigma}$, which is difficult to compute directly from quantities defined only at moving particles. MPM avoids this by solving the weak form of the momentum equation, obtained by testing against an arbitrary virtual velocity field $\delta\mathbf{v}$ and integrating by parts:

$$\begin{aligned}\int_\Omega \rho\,\mathbf{a}\cdot\delta\mathbf{v}\,d\Omega + \int_\Omega \boldsymbol{\sigma} : \nabla\delta\mathbf{v}\,d\Omega \\ = \int_\Omega \mathbf{f}_b\cdot\delta\mathbf{v}\,d\Omega + \int_\Gamma \mathbf{t}\cdot\delta\mathbf{v}\,d\Gamma.\end{aligned} \tag{2}$$

The four terms represent, respectively: inertial virtual work, internal virtual work from stress, external virtual work from body forces, and virtual work from boundary tractions $\mathbf{t}$. This is the Principle of Virtual Work: for every admissible $\delta\mathbf{v}$, inertial and internal work must balance external work. Critically, the stress divergence has been eliminated in favor of $\boldsymbol{\sigma} : \nabla\delta\mathbf{v}$, which requires only first derivatives of the test

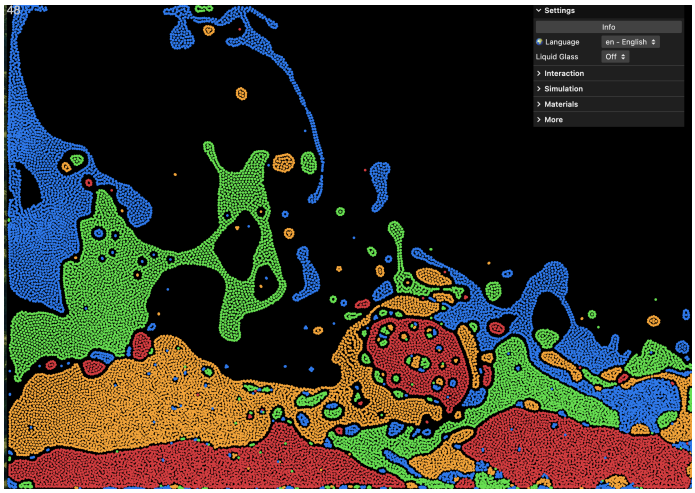

Figure 1: A screenshot of Grant Kot's Liquid Layers simulation during a mouse interaction. The circular cluster near the cursor illustrates coherent multi-material motion under external forcing. In undisturbed regions, clear density stratification is visible, with heavier materials settling beneath lighter ones. Interfaces remain sharp at large scales but exhibit small-scale mixing due to advection and numerical diffusion. The stretched filaments and detached droplets demonstrate complex flow behavior, including fragmentation and weak interfacial cohesion, highlighting challenges in multi-material MPM such as interface preservation and stable particle–grid coupling.

function rather than derivatives of the stress field itself.

MPM discretizes this statement by using particles as quadrature points, $\int_\Omega f(\mathbf{x})\,d\Omega \approx \sum_p V_p f(\mathbf{x}_p)$, and using grid basis functions $N_i(\mathbf{x})$ as test functions. Choosing $\delta\mathbf{v} = N_i(\mathbf{x})$ yields a nodal equation of motion $m_i\mathbf{a}_i = \mathbf{f}_i^{\text{int}} + \mathbf{f}_i^{\text{ext}}$, where

$$\begin{aligned} m_i &= \sum_p m_p N_i(\mathbf{x}_p), \\ \mathbf{f}_i^{\text{int}} &= -\sum_p V_p \boldsymbol{\sigma}_p \nabla N_i(\mathbf{x}_p), \\ \mathbf{f}_i^{\text{ext}} &= \sum_p m_p N_i(\mathbf{x}_p)\,\mathbf{g}. \end{aligned} \tag{3}$$

Particles carry mass, deformation history, and material state; the grid computes spatial derivatives and updates momentum. This hybrid structure is the central idea of MPM: particles handle large deformation naturally, while the grid provides a clean framework for computing gradients and enforcing boundary conditions.

MLS-MPM and Grid Transfers

The key computational issue in MPM is transferring information between particles and grid nodes. Standard transfers represent particle velocity as a single constant value, discarding information about local velocity variation. The Affine Particle-In-cell (APIC) method [4] improves this by storing an affine velocity matrix $\mathbf{C}_p$ on each particle, so that the local velocity field is approximated as $\mathbf{v}(\mathbf{x}) \approx \mathbf{v}_p + \mathbf{C}_p(\mathbf{x} - \mathbf{x}_p)$. Each particle thus encodes not only its translational velocity but also a local linear approximation capturing rotation, shear, and stretch. This reduces numerical dissipation and improves angular momentum conservation compared with constant-velocity transfers.

MLS-MPM is a formulation of MPM derived from a moving least-squares velocity approximation that leverages APIC. Its practical advantage is that the APIC affine transfer and stress contribution can be combined efficiently in a single particle-to-grid scatter step. MLS-MPM improves transfers by using APIC-style affine velocity fields and quadratic B-spline kernels, producing more accurate and less dissipative transfers than constant-velocity particle methods as in the canonical MPM formulation.

Grid-particle transfers use interpolation weights from a kernel function. We use the quadratic B-spline kernel, which in one dimension, for normalized distance $r = |x_i - x_p|/\Delta x$, is

$$N(r) = \begin{cases} \frac{3}{4} - r^2, & 0 \le r < \frac{1}{2}, \\ \frac{1}{2}\left(\frac{3}{2} - r\right)^2, & \frac{1}{2} \le r < \frac{3}{2}, \\ 0, & r \ge \frac{3}{2}. \end{cases} \tag{4}$$

In two dimensions, the weight is the tensor product $w_{ip} = N(r_x)N(r_y)$. Because the quadratic B-spline has support over $\frac{3}{2}$ grid cells in each dimension, each particle interacts with a 3×3 stencil of nearby nodes in 2D. Its piecewise-linear derivative ∇w_{ip} is used to compute stress forces in P2G.

MLS-MPM Algorithm

Each time step proceeds through six stages: particle-to-grid transfer, grid update, grid-to-particle transfer, deformation gradient update, particle advection, and grid reset.

1. Particle-to-Grid (P2G) Transfer

Particles scatter mass and momentum to nearby grid nodes. The nodal mass is $m_i = \sum_p w_{ip}\, m_p$. With APIC, each particle contributes affine momentum rather than only constant momentum:

$$(m\mathbf{v})_i = \sum_p w_{ip}\, m_p \left[\mathbf{v}_p + \mathbf{C}_p(\mathbf{x}_i - \mathbf{x}_p)\right]. \tag{5}$$

The affine correction $\mathbf{C}_p(\mathbf{x}_i - \mathbf{x}_p)$ preserves local velocity variation such as spin or shear across the transfer. The grid velocity is then $\mathbf{v}_i = (m\mathbf{v})_i/m_i$.

Internal forces from stress are also transferred. From the weak form, $\mathbf{f}_i^{\text{int}} = -\sum_p V_p^0\, \boldsymbol{\tau}_p\, \nabla w_{ip}$, where $\boldsymbol{\tau}_p = J_p\boldsymbol{\sigma}_p$ is the Kirchhoff stress, V_p^0 is the reference volume, and $J_p = \det \mathbf{F}_p$. For a hyperelastic material, the first Piola–Kirchhoff stress is $\mathbf{P}_p = \partial\Psi(\mathbf{F}_p)/\partial\mathbf{F}_p$, giving $\boldsymbol{\tau}_p = \mathbf{P}_p\mathbf{F}_p^T$. In MLS-MPM, the stress contribution is folded into the momentum scatter as a single fused operation:

$$(m\mathbf{v})_i \mathrel{+}= w_{ip}\left[m_p\mathbf{v}_p + \left(m_p\mathbf{C}_p - \Delta t\, V_p^0\, \mathbf{P}_p\mathbf{F}_p^T\right)(\mathbf{x}_i - \mathbf{x}_p)\right]. \tag{6}$$

The first term transfers particle momentum; the stress term modifies grid momentum to account for internal material forces, encoding the effects of compression, tension, and shear.

2. Grid Velocity Update

Once mass and momentum have been accumulated at each node with $m_i > 0$, grid velocities are updated by Newton's second law:

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^n + \Delta t\left(\frac{\mathbf{f}_i^{\text{int}}}{m_i} + \mathbf{g} + \frac{\mathbf{f}_i^{\text{ext}}}{m_i}\right), \tag{7}$$

where $\mathbf{f}_i^{\text{ext}}$ collects any forces not originating from the constitutive model, such as user-applied mouse forces. When using the fused MLS-MPM P2G form, stress is already incorporated in the momentum scatter, so this step adds only external forces and boundary conditions. Boundary conditions are enforced by modifying grid node velocities; for example, at a solid wall with outward normal $\mathbf{n}$, any velocity component pointing into the wall is zeroed: if $\mathbf{v}_i \cdot \mathbf{n} < 0$, then $\mathbf{v}_i \leftarrow \mathbf{v}_i - (\mathbf{v}_i \cdot \mathbf{n})\mathbf{n}$.

3. Grid-to-Particle (G2P) Transfer

Each particle gathers updated velocity information from nearby grid nodes: $\mathbf{v}_p^{n+1} = \sum_i w_{ip}\, \mathbf{v}_i^{n+1}$. The APIC affine matrix is updated by fitting the local grid velocity field:

$$\mathbf{C}_p^{n+1} = \frac{4}{\Delta x^2}\sum_i w_{ip}\, \mathbf{v}_i^{n+1}\,(\mathbf{x}_i - \mathbf{x}_p)^T. \tag{8}$$

The factor $4/\Delta x^2$ is the inverse of the quadratic B-spline kernel's weighted second moment in the MLS-MPM dis-

cretization. The particle recovers both its average velocity and a local velocity gradient: $\mathbf{v}_p$ is used to advect the particle, while $\mathbf{C}_p$ preserves local affine motion for the next P2G transfer.

4. Deformation Gradient Update

The deformation gradient $\mathbf{F}_p$ tracks how an infinitesimal material element around particle p has deformed relative to its reference configuration. It evolves as $D\mathbf{F}/Dt = (\nabla \mathbf{v})\mathbf{F}$, which is integrated explicitly. In MLS-MPM/APIC, $\mathbf{C}_p$ approximates the local velocity gradient, so the update is simply

$$\mathbf{F}_p^{n+1} = \left(\mathbf{I} + \Delta t\, \mathbf{C}_p^{n+1}\right) \mathbf{F}_p^n. \tag{9}$$

Maintaining an accurate $\mathbf{F}_p$ is essential because the constitutive model uses it to compute $\boldsymbol{\sigma}_p$ at the next time step.

5. Particle Advection

Particles are advected using their updated velocities: $\mathbf{x}_p^{n+1} = \mathbf{x}_p^n + \Delta t\, \mathbf{v}_p^{n+1}$. Particle-level boundary conditions, such as clamping positions to the interior of the domain, are applied here. Particles are the persistent material samples: unlike the grid, they survive every time step, continuously carrying mass, velocity, deformation, and any other material history.

6. Grid Reset

After all particle quantities have been updated, the grid is cleared: $m_i \leftarrow 0$, $(m\mathbf{v})_i \leftarrow \mathbf{0}$, $\mathbf{f}_i \leftarrow \mathbf{0}$. The grid is not a persistent mesh but a temporary computational scratchpad. Discarding and rebuilding it each step is what allows MPM to handle arbitrarily large deformations without mesh distortion.

2 Fluid Simulation Implementation

I implemented a small 2D weakly-compressible MLS-MPM liquid solver with grid-space mouse forcing, as shown in Fig. 2. Implementation details and limitations are discussed in the following sections.

Constitutive Model

The solver uses a weakly-compressible Newtonian fluid constitutive model. The Kirchhoff stress (used in the P2G scatter, Eq. (6)) is

$$\boldsymbol{\tau} = -Jp\mathbf{I} + J\mu(\mathbf{C} + \mathbf{C}^\top),$$

where J is a scalar volume-change proxy (discussed below), $\mathbf{C}$ is the APIC affine velocity matrix (which encodes the local velocity gradient), μ is the dynamic viscosity, and p is determined by the Tait equation of state:

$$p = K\left(J^{-\gamma} - 1\right).$$

Here K is the bulk modulus and γ is the isentropic Tait exponent. The first term $-Jp\mathbf{I}$ is the isotropic pressure response; the second term $J\mu(\mathbf{C} + \mathbf{C}^\top)$ is the Newtonian viscous stress, with $\frac{1}{2}(\mathbf{C} + \mathbf{C}^\top)$ approximating the rate-of-deformation tensor.

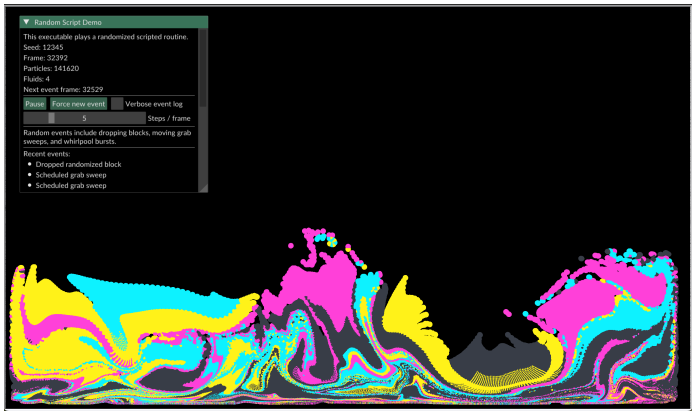

Figure 2: A snapshot of the MPM-based multi-material liquid layers under randomized forcing (block drops and whirlpool/grab interactions). Distinct materials remain visible but undergo extreme deformation, producing folded interfaces, thin filaments, and shear-driven mixing. Large-scale vortical motion emerges while small-scale diffusion blurs interfaces, illustrating both the strengths and limitations of MPM in preserving material boundaries under strong advection. See recorded demos here.

This is implemented in `kirchhoffStress()`, which clamps $J \in [0.1, 5]$, evaluates the Tait pressure, applies a small tension floor $p \geq -0.1K$ to prevent unphysical tensile blow-up during expansion, and assembles the two stress contributions.

Enforcing (Near-)Incompressibility

Truly incompressible flow requires $\nabla \cdot \mathbf{u} = 0$, which can be enforced in MPM either via a pressure Poisson solve on the grid or by using a sufficiently stiff equation of state. This implementation uses the latter (weakly-compressible) approach: by choosing K large, density fluctuations are penalized and the fluid behaves approximately as incompressible. The trade-off is that larger K raises the material wave speed $c_0 = \sqrt{K\gamma/\rho_0}$, tightening the CFL stability condition and requiring smaller Δt.

At initialization, the code computes and reports a CFL-suggested timestep $\Delta t_{\text{CFL}} = 0.3\,\Delta x / c_0$ based on the material wave speed. This estimate is derived purely from the acoustic CFL condition for the pressure wave; it does not account for the explicit viscous update, the linearization error in the J update, or the grid-velocity modifications from mouse forcing. In practice, Δt_{CFL} is often close to the stability boundary, and the simulation frequently explodes when it is used directly. The default timestep is therefore set conservatively below Δt_{CFL}, providing a safety margin against these additional sources of instability.

The choice of K involves a direct stability trade-off. If K is too large relative to Δt, the pressure response to small compressions becomes so stiff that the explicit integrator overshoots, causing J to oscillate and pressure to spike—leading to an explosion. Conversely, if K is too small, the fluid becomes noticeably compressible: particles clump and spread without resistance, and the simulation looks more

like a gas than a liquid.

Algorithmic Simplifications and Deviations from MLS-MPM

One complete physics timestep, executed by `step()`, follows the MLS-MPM algorithm [3] with several modifications. The UI exposes a substepping parameter that runs multiple calls to `step()` per rendered frame, allowing the user to trade compute time for stability when using large K, small Δt, or aggressive mouse interaction.

1. **Separated P2G passes.** Standard MLS-MPM fuses mass/momentum and stress transfer into a single P2G scatter (Eq. (6)). Here they are split into two passes—`P2G_mass()` and `P2G_stress()`—with an intermediate volume recomputation step between them (see below).

2. **Scalar J update.** For a fluid, the constitutive model depends only on the scalar $J = \det(\mathbf{F})$; the full deformation gradient $\mathbf{F}$ update of Eq. (9) need not be stored or performed. After gathering grid velocities back to particles via G2P (Eq. (8)), J is updated as

$$J_p^{n+1} = J_p^n (1 + \Delta t \, \mathrm{tr}(\mathbf{C}_p)) \,,$$

where $\mathrm{tr}(\mathbf{C}_p) \approx \nabla \cdot \mathbf{v}_p$ is the local volumetric strain rate. This is a linearized (first-order Euler) discretization of $\dot{J} = J \nabla \cdot \mathbf{v}$, replacing the matrix update of Eq. (9). The linearization is not positivity-preserving, so J is clamped to $[0.1, 5]$ after each update.

3. **Per-step volume recomputation (`volumeRecompute`).** Rather than propagating $\mathrm{vol}_p = J_p \, \mathrm{vol}_p^0$ from the initial seeding, the effective particle volume is recomputed each step from the grid mass field (using the nodal masses from Eq. (3)):

$$\rho_p = \sum_i w_{ip} \frac{m_i}{\Delta x^2}, \qquad \mathrm{vol}_p = \frac{m_p}{\rho_p}.$$

This decouples the volume estimate from accumulated drift in J and corrects for the free-surface artifact in which sparse particles near the surface underestimate their volume and generate spurious pressure. The density is floored at $0.1\rho_0$ to avoid division by zero at empty nodes.

APIC velocity transfer. Both P2G and G2P use the Affine Particle-in-Cell (APIC) scheme [4] as in Eqs. (6) and (8). In P2G, node momentum receives an affine correction $\mathbf{C}_p(\mathbf{x}_i - \mathbf{x}_p)$ that exactly recovers linear velocity fields, reducing the numerical diffusion of standard PIC. In G2P, the affine matrix is updated as $\mathbf{C}_p = D^{-1}\mathbf{B}_p$ where $\mathbf{B}_p = \sum_i w_{ip} \, \mathbf{v}_i^{\mathrm{new}} \otimes (\mathbf{x}_i - \mathbf{x}_p)$ and $D^{-1} = 4/\Delta x^2$ is the quadratic B-spline normalization constant.

Boundary conditions. `gridUpdate()` (Eq. (7)) applies one-sided slip conditions at the left, right, and bottom walls by zeroing the penetrating normal velocity component at nodes within a two-cell buffer of each wall; the top boundary is left open. Particles are additionally clamped to the domain interior in `G2P_advect()`.

Particle seeding. Each rectangular block is seeded with $N_{\mathrm{ppc}} \times N_{\mathrm{ppc}}$ particles per grid cell, placed at sub-cell centers with a small deterministic jitter ($\pm 15\%$ of sub-cell size) to break lattice symmetry. Particle mass is set to $m_p = \rho_0 \, \Delta x_p \, \Delta y_p$, which ensures $J = 1$ initially and produces zero Tait pressure at the start of the simulation. To initialize a Rayleigh–Taylor unstable configuration, the denser fluid is assigned to the upper band of the `BlockSpec` (via `top_fluid`) and the lighter fluid to the lower band (`bottom_fluid`); the small seeding jitter is sufficient to seed the interface perturbation.

Mouse-Force Mathematics

Mouse forces are applied on the grid inside `applyMouseForcesToGrid()`, after the grid velocity update (Eq. (7)) and before G2P, so particles feel the interaction only through the grid. The simulation exposes two modes.

- **Grab mode.** For each grid node i within radius R of the cursor at $\mathbf{x}_{\mathrm{mouse}}$, let $\mathbf{r} = \mathbf{x}_i - \mathbf{x}_{\mathrm{mouse}}$ and $d = |\mathbf{r}|$. A compact falloff weight $w(d) = (1 - (d/R)^2)^2$ gives full influence at the cursor center and smoothly tapers to zero at the disc boundary. The inward unit direction $\hat{\mathbf{e}}_r = -\mathbf{r}/d$ points from each node toward the cursor, so the velocity increment

$$\Delta \mathbf{v}_i = w(d) \cdot s \cdot \Delta t \cdot \hat{\mathbf{e}}_r$$

pulls all nodes within the disc toward the cursor with strength proportional to s and their proximity to it. This acts as a direct velocity injection (an impulse-like attractor field), not a conservative force; there is no restoring term and no target velocity, so the fluid accelerates toward the cursor for as long as the button is held.

- **Whirlpool mode.** For nodes inside radius R, let $q = 1 - d/R$ and define the tangential and inward unit vectors $\hat{\mathbf{e}}_t = (-r_y, r_x)/d$ and $\hat{\mathbf{e}}_r = -\mathbf{r}/d$. A target velocity field is constructed as

$$\mathbf{v}_{\mathrm{target}} = \underbrace{\omega \, d}_{\text{solid-body spin}} \hat{\mathbf{e}}_t + \underbrace{0.6 \, \omega \, q}_{\text{inward pull}} \hat{\mathbf{e}}_r,$$

where ω is the user strength parameter, ωd produces a solid-body rotation profile (tangential speed proportional to radius), and $0.6\,\omega\,q$ is an inward radial speed that peaks at the cursor and vanishes at the boundary of the influence disc. Rather than adding a fixed $\Delta \mathbf{v}$ as in grab mode, grid velocities are relaxed toward $\mathbf{v}_{\mathrm{target}}$ each step:

$$\mathbf{v}_i^{\mathrm{new}} \leftarrow \mathbf{v}_i^{\mathrm{new}} + \gamma \, q^2 \, (\mathbf{v}_{\mathrm{target}} - \mathbf{v}_i^{\mathrm{new}}) \, \Delta t, \quad \gamma = 8.$$

The factor $q^2 = (1 - d/R)^2$ provides a smooth spatial falloff, and the fixed gain $\gamma = 8$ controls how quickly grid velocities are driven toward the target. This relaxation formulation prevents the abrupt velocity jumps that a direct injection would produce near the disc boundary, and naturally limits the maximum induced speed to $|\mathbf{v}_{\mathrm{target}}|$ rather than accumulating without bound.

Limitations and Stability

- **Weakly-compressible model and bulk modulus sensitivity.** The simulation is not truly incompressible. Increasing K suppresses density fluctuations but raises c_0 and tightens the CFL constraint; if Δt is not reduced accordingly, the explicit pressure update overshoots and the simulation explodes. Decreasing K too far makes the fluid visibly compressible—particles clump and spread without resistance, and density waves become apparent.

- **CFL estimate does not guarantee stability.** The reported $\Delta t_{\text{CFL}} = 0.3\,\Delta x/c_0$ is derived solely from the acoustic wave speed of the pressure model. It does not account for the explicit viscous update, accumulated drift in the linearized J update (Simplification 2 above), or the nonphysical velocity injections from mouse forcing—all of which impose additional, parameter-dependent stability constraints. As a result, running at Δt_{CFL} frequently causes the simulation to explode, and the default timestep is set conservatively below it.

- **Viscosity and timestep coupling.** The viscous stress $J\mu(\mathbf{C}+\mathbf{C}^\top)$ is evaluated explicitly. Very large μ or Δt can cause the viscous forces to overshoot, leading to oscillatory or divergent behavior. In practice, μ and Δt must be kept in a compatible range.

- **Mouse-force instability.** Because mouse interaction directly modifies grid velocities after the update of Eq. (7), it can inject arbitrary energy into the simulation. Aggressive grab or whirlpool strokes—particularly with large s or large Δt—can produce velocities large enough to violate CFL, causing J to drift outside its clamped range and pressure to spike, leading to an explosion of particles. Substepping reduces this risk by shrinking the per-step Δt.

- **Linearized J update.** The update $J \leftarrow J(1+\Delta t\,\nabla\cdot\mathbf{v})$ (Simplification 2) is first-order accurate and not positivity-preserving. Accumulated drift in J is the primary driver of pressure error over long simulations; the volume recomputation step (Simplification 3) partially mitigates this but does not eliminate it.

- **MPM numerical diffusion and lack of stratification.** The P2G/G2P interpolation steps act as a smoothing operator on the velocity and mass fields. At interfaces between two fluids, this diffusion numerically mixes densities and momenta across the background grid. Particles of different fluids that share a grid cell are effectively averaged together during each transfer, so even an initially unstable Rayleigh–Taylor configuration does not stratify sharply: the interface is continuously smeared rather than advected as a sharp front. Subgrid-scale interactions between nearby particles of different types are not resolved.

- **Simple boundary conditions.** Wall treatment is basic nodal velocity clipping. Physical accuracy near walls and splash behavior are resolution-dependent.

References

[1] G. Kot, "Liquid Layers," interactive demo, 2013. Available: https://grantkot.com/ll/

[2] S. Clavet, P. Beaudoin, and P. Poulin, "Particle-based viscoelastic fluid simulation," in *Proc. ACM SIGGRAPH/Eurographics Symp. Computer Animation*, 2005, pp. 219–228.

[3] Y. Hu, Y. Fang, Z. Ge, Z. Qu, Y. Zhu, A. Pradhana, and C. Jiang, "A moving least squares material point method with displacement discontinuity and two-phase liquid inclusion," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 150:1–150:14, 2018.

[4] C. Jiang, C. Schroeder, A. Selle, J. Teran, and A. Stomakhin, "The affine particle-in-cell method," *ACM Transactions on Graphics*, vol. 34, no. 4, pp. 51:1–51:10, 2015.